

Contents

Compatibility contract	1
What this means concretely	1
What this rules out	1
What this rules in	1
Storage-type semantics	2
How the round-trip works	2
Where this diverges from Stata	2
Documented no-op options	2
Current tea-extensions	3
Reporting compatibility breaks	3

Compatibility contract

tea is a proper subset of Stata.

Every tea do-file must run unchanged in Stata. This is the project’s primary design rule and it takes precedence over every other consideration including elegance, ergonomics, and consistency.

What this means concretely

- Every tea command, option, and function — if it exists in Stata, must behave identically. If it does not exist in Stata, it must not exist in tea, or it must be opt-in via the `--tea-extensions` flag.
- Default runtime mode is `--strict-stata`. Anything Stata wouldn’t accept is rejected.
- The `.dta` format is the supported exchange format. Tea reads and writes Stata 14+ (.dta format 118).
- Numerical output should match Stata to at least 6 digits on coefficients and 4 digits on standard errors.

What this rules out

- “Tea has a better way to do this” — if it conflicts with Stata, it’s out, even if Stata’s behavior is awkward.
- Convenience extensions that aren’t gated behind `--tea-extensions`.
- Silently accepting commands or options Stata wouldn’t accept.
- Output formats that don’t match Stata’s (table headers, column widths, statistical reporting conventions).

What this rules in

- Implementations more efficient than Stata (different algorithm, same result) are fine.
- Bug-for-bug compatibility is NOT required — if Stata has a known bug, tea may produce the correct result. This is the one explicit exception.
- Performance, error messages, REPL ergonomics may differ as long as the observable behavior on do-files matches.

Storage-type semantics

Stata has five numeric storage types (byte, int, long, float, double) and per-variable width. Tea stores all numeric values internally as IEEE double regardless of declared type. This is a **deliberate design decision** locked in v0.1; it simplifies the engine and is invisible to users in almost all cases.

How the round-trip works

- **Reading .dta:** every numeric storage type (byte/int/long/float/ double) is upcast to double on load. No value changes — the bit patterns of float and the integer ranges of byte/int/long all fit losslessly in double. This costs RAM relative to Stata’s native representation, but no precision.
- **Writing .dta:** every numeric column is per-column compressed to the smallest Stata type that fits its actual values losslessly, matching what Stata’s own `compress` command would produce. Files saved by tea open in Stata with the same storage types they’d have if a Stata user had run `compress` before save.
- **In-memory:** the type qualifier on `gen byte x = ...` is accepted but is essentially a hint that affects nothing at runtime. The column is double; on save, if every value of `x` fits in a byte, it will be written as a byte.

Where this diverges from Stata

In strict Stata, declaring `gen byte x = 1` and then attempting `replace x = 999` is an error (999 exceeds byte range). In tea, the declared type is advisory, so the `replace` succeeds; on save, the column is written as the smallest type that fits 999 (int). This is one observable behavioral divergence from Stata, in the direction of permissiveness.

Storage-type strictness will be available as a `--tea-extensions` opt-in in a future release if there’s demand. In the meantime, strict storage-type enforcement is documented as a known divergence, not a bug.

Documented no-op options

These Stata options are accepted by tea but have no effect, because the underlying behavior they control isn’t relevant in tea’s environment:

- `set more on/off` — pagination. Tea doesn’t paginate output; rely on your terminal’s scrollbar or `log using FILE` to capture output for later review. Modern terminals make pagination obsolete.
- `set linesize N` — line wrapping width. Tea doesn’t wrap; if you want narrower output, resize your terminal.
- `set matsize N` — maximum matrix size. Tea has no fixed matrix-size limit.
- `set type float|double` — default storage type for new numeric variables. Tea stores all numerics as double regardless.
- `compress` — Stata’s `compress` reduces in-memory storage by picking the smallest type per column. Tea is always uncompressed (double) in memory; on save, columns are automatically compressed. So `compress` issued by the user is a no-op.
- `recast TYPE varlist` — change the storage type of existing variables. Tea’s storage is type-agnostic; the declared type is updated but the in-memory representation does not change.

Each of these can be set in a do-file without consequence; the operation is silently a no-op.

Current tea-extensions

The following are accepted only with `--tea-extensions`. All produce a clear “tea extension” error message in strict-stata mode (the default).

- `mkdir DIR, recursive` (and the `p` alias) — creates intermediate parents like `mkdir -p`. Stata’s `mkdir` creates only one level.

This list is intentionally short — most things you might expect to be tea-extensions are actually documented Stata commands (`dir`/`ls`/`rm` are all official Stata, despite their Unix flavor).

Reporting compatibility breaks

If you find a tea behavior that differs from Stata, file a bug. Include:

- The shortest do-file that reproduces the divergence.
- The Stata output for the same input.
- The tea output.

These are treated as bugs, not feature requests.